📘 Week 1 - Day 2: Contexts, Inputs & Step Summaries

Production Pipelines • Dynamic Metadata • Interactive UI Control • Live Markdown Dashboards

1. The Secret Brain: GitHub Context Objects

When any GitHub Actions runner boots up, GitHub automatically populates a metadata dictionary called **Context Objects**. These provide dynamic details about the build, author, environment, and code commit.

```
[ GitHub Engine ] ──► Injects Context Objects into Runner Memory
                                  │
   ├── ${{ github.actor }}       ──► Username who initiated the workflow
   ├── ${{ github.sha }}         ──► 40-character commit hash (e.g. 9e7a0fb... )
   ├── ${{ github.ref_name }}    ──► Active Git branch (e.g. main)
   ├── ${{ github.event_name }}  ──► Trigger source (e.g. push vs workflow_dispatch)
   └── ${{ runner.os }}          ──► Host OS (Linux, Windows, macOS)
```

Production Use Cases (9 LPA Standard):

Context Variable	Real-World DevOps Application
`${{ github.sha }}`	Tags Docker container images uniquely (`docker build -t app:${{ github.sha }} .`) so any live deployment can be traced to exact source code.
`${{ github.actor }}`	Sends targeted Slack/Discord alerts mentioning the exact engineer who triggered or broke a deployment.
`${{ github.ref_name }}`	Guards production releases so code from experimental branches never deploys to production servers.

2. Interactive UI Inputs (`workflow_dispatch.inputs`)

DevOps engineers build self-service portals inside GitHub Actions by defining `inputs` under the `workflow_dispatch` trigger. This renders real UI controls in the GitHub browser tab.

```yaml
on:
  workflow_dispatch:
    inputs:
      target_env:
        description: 'Select Target Environment'
        required: true
        default: 'staging'
        type: choice
        options:
          - development
          - staging
          - production
      deployer_name:
        description: 'DevOps Lead / Author'
        required: true
        default: 'Sanjit'
        type: string
```

> 💡 **How to Access Inputs in Steps:**
> Access any input anywhere in your workflow steps using: `${{ inputs.target_env }}` and `${{ inputs.deployer_name }}`.

3. Live Visual Dashboards (`$GITHUB_STEP_SUMMARY`)

Instead of forcing engineering managers to dig through raw terminal logs, production workflows output formatted Markdown tables directly to **`$GITHUB_STEP_SUMMARY`**.

```
      - name: Generate Live Summary Card
        run: |
          echo "## 🚀 Deployment Status Report" >> $GITHUB_STEP_SUMMARY
          echo "" >> $GITHUB_STEP_SUMMARY
          echo "| Metric | Value |" >> $GITHUB_STEP_SUMMARY
          echo "| :--- | :--- |" >> $GITHUB_STEP_SUMMARY
          echo "| 👨‍💻 **DevOps Engineer** | ${{ inputs.deployer_name || github.actor }} |" >> $GITH
          echo "| 🎯 **Target Environment** | \`${{ inputs.target_env || 'auto-push' }}\` |" >> $(
          echo "| 🌿 **Git Branch** | \`${{ github.ref_name }}\` |" >> $GITHUB_STEP_SUMMARY
          echo "| 📌 **Commit SHA** | \`${{ github.sha }}\` |" >> $GITHUB_STEP_SUMMARY
          echo "| 🧪 **Test Suite** | ✅ PASSED (100%) |" >> $GITHUB_STEP_SUMMARY
          echo "| 🚀 **Status** | **SUCCESSFULLY VERIFIED** |" >> $GITHUB_STEP_SUMMARY
```

4. Complete Day 2 Master Workflow Code

```yaml
name: 🚀 Live Deployment Control Center

on:
  push:
    branches: [ main ]
  workflow_dispatch:
    inputs:
      target_env:
        description: '🎯 Select Target Environment'
        required: true
        default: 'staging'
        type: choice
        options:
          - development
          - staging
          - production
      deployer_name:
        description: '✍️ Enter Your Name / Lead'
        required: true
        default: 'Sanjit'
        type: string

jobs:
  release_control:
    name: ⚡ Release & Deploy Engine
    runs-on: ubuntu-latest

    steps:
      - name: 1. Checkout Code
        uses: actions/checkout@v4

      - name: 2. Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: 3. Install Dependencies
        run: npm ci

      - name: 4. Run Automated Tests
        run: npm test

      - name: 5. Generate Live Summary Card
        run: |
          echo "## 🚀 Deployment Status Report" >> $GITHUB_STEP_SUMMARY
          echo "| Metric | Value |" >> $GITHUB_STEP_SUMMARY
          echo "| :--- | :--- |" >> $GITHUB_STEP_SUMMARY
          echo "| 👨‍💻 **DevOps Engineer** | ${{ inputs.deployer_name || github.actor }} |" >> $GITH
          echo "| 🎯 **Target Environment** | \`${{ inputs.target_env || 'auto-push' }}\` |" >> $(
```

5. 9 LPA Interview Questions for Day 2

Q1: How do you build an on-demand deployment pipeline where QA can choose between Staging and Production from GitHub UI?
Answer: We configure the `workflow_dispatch` event trigger with `inputs` of type `choice` listing the available environments. In the steps, we read `${{ inputs. }}` to route the deployment target.

Q2: What is `$GITHUB_STEP_SUMMARY` and why is it preferred in production?
Answer: `$GITHUB_STEP_SUMMARY` is an environment variable pointing to a special markdown buffer on the runner. Any markdown appended to it renders as a visual report on the Actions summary page, providing instant status metrics without parsing terminal logs.